

Αλγόριθμοι και Πολυπλοκότητα Πρώτη Σειρά Γραπτών Ασκήσεων

Παναγή Αντρέας

Π.Μ.Σ. - ΑΛΜΑ

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα *

Νοέμβριος 2024



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

* Στοιχειοθέτηση με χρήση $\text{\LaTeX}$

Περιεχόμενα

Άσκηση 1: Υπολογισμός Κυρίαρχων Θέσεων	1
Εκφώνηση	1
Λύση	1
Διαισθητική εξήγηση	1
Αλγόριθμος	1
Πολυπλοκότητα	1
Απόδειξη ορθότητας	2
Άσκηση 2: Επιλογή	3
Εκφώνηση	3
Λύση: Ερώτημα α)	3
Διαισθητική εξήγηση	3
Αλγόριθμος	3
Ορθότητα	4
Πολυπλοκότητα	4
Λύση: Ερώτημα β)	4
Παρατηρήσεις - Μεθοδολογία	4
Αλγόριθμος	4
Πολυπλοκότητα	4
Ορθότητα	5
Άσκηση 3: Άθροισμα Στοιχείων Υποσυνόλων και Υπακολουθιών	6
Εκφώνηση	6
Λύση: Ερώτημα α)	6
Περιγραφή	6
Αλγόριθμος	6
Πολυπλοκότητα	7
Ορθότητα	7
Λύση: Ερώτημα β)	7
Περιγραφή	7
Αλγόριθμος	7
Πολυπλοκότητα	8
Ορθότητα	8
Άσκηση 4: Μη Επικαλυπτόμενα Διαστήματα Μέγιστου Συνολικού Μήκους	9
Εκφώνηση	9
Λύση: Ερώτημα 1.	9
Κριτήριο: μέγιστο μήκος	9
Κριτήριο: νωρίτερη περάτωση	9
Λύση: Ερώτημα 2.	9
Περιγραφή - Διαισθητική προσέγγιση	9
Αλγόριθμος 1	10
Αλγόριθμος 2	10
Πολυπλοκότητα	10
Ορθότητα	10

Άσκηση 5: Θέσεις Στάθμευσης	12
Εκφώνηση	12
Λύση: Ερώτημα 1.	12
Σκιαγράφηση Απόδειξης	12
Αλγόριθμος	12
Πολυπλοκότητα	13
Ορθότητα	13
Λύση: Ερώτημα 2.	13
Περιγραφή	13
Αλγόριθμος	14
Πολυπλοκότητα	14
Ορθότητα	14

Άσκηση 1: Υπολογισμός Κυρίαρχων Θέσεων

Εκφώνηση

Θεωρούμε πίνακα $A[1..n]$ με n φυσικούς αριθμούς. Για κάθε $i = 2, \dots, n$, η θέση που κυριαρχεί της θέσης i στον πίνακα A είναι η πλησιέστερη θέση που προηγείται της i και η τιμή της ξεπερνά την τιμή $A[i]$. Τυπικά, θεωρώντας ότι $A[0] = \infty$, η θέση που κυριαρχεί της θέσης i στον A είναι η μέγιστη θέση j , $0 \leq j < i$, για την οποία ισχύει $A[j] > A[i]$ (βλ. ότι η θέση που κυριαρχεί της 1 είναι η 0). Να διατυπώσετε αποδοτικό αλγόριθμο που υπολογίζει τη θέση που κυριαρχεί της θέσης i , για κάθε $i = 1, \dots, n$, στον πίνακα $A[1..n]$. Να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα του αλγορίθμου σας.

Λύση

Διαισθητική εξήγηση

Η ιδέα του αλγορίθμου είναι να τροποποιήσουμε τον αλγόριθμο *Merge Sort* ώστε παράλληλα με την ταξινόμηση να κατασκευάσουμε και τον πίνακα K .

Η διαίσθηση μας, μας οδηγεί στην τροποποίηση του αλγορίθμου *Merge Sort* αφού για κάθε θέση i του αρχικού πίνακα πρέπει να συγκρίνουμε τα στοιχεία που βρίσκονται πριν από την θέση αυτή και να κρατήσουμε την μεγαλύτερη θέση j η οποία περιέχει στοιχείο $A[j] > A[i]$ (αλλιώς κρατάμε το 0 με την παραδοχή ότι $A[0] = \infty$).

Ταξινομώντας κατά αύξουσα σειρά, αν υπάρχει τέτοιο στοιχείο, τότε αυτό σε κάποια φάση θα συγκριθεί με το $A[i]$ ώστε να μετακινηθεί δεξιότερα (όπως κάθε αριστερότερο μεγαλύτερο στοιχείο). Έτσι εγγυόμαστε ότι αν υπάρχει τέτοιο στοιχείο, σε κάποια φάση ο αλγόριθμος θα το εξετάσει και το μόνο που έχουμε να κάνουμε είναι να συγκρίνουμε την αρχική του θέση με τα προηγούμενα αριστερότερα μεγαλύτερα που βρήκαμε (και να κρατήσουμε την μεγαλύτερη θέση).

Αλγόριθμος

1. Αρχικοποιούμε ένα πίνακα $K[1, \dots, n] = [0, \dots, 0]$
Όπου το $K[i]$ αντιστοιχεί στην θέση που κυριαρχεί την θέση i του πίνακα A , με $i = 1, \dots, n$.
2. Δημιουργούμε ένα νέο πίνακα ο οποίος αποτελείται από ζεύγη κάθε στοιχείου και της θέσης του. Δηλαδή, $M = [(A[1], 1), \dots, (A[n], n)]$
3. Εκτελούμε ταξινόμηση *Merge Sort* κατά αύξουσα σειρά, ως προς την πρώτη συντεταγμένη του ζεύγους
(α') Τροποποιούμε την υπο-ρουτίνα *Merge* ώστε μετά από κάθε σύγκριση $A[i], A[j]$ να ελέγχει: αν η συντεταγμένη της θέσης του μεγαλύτερου (χ.β.γ. έστω $A[i]$) είναι μικρότερη από την θέση του άλλου ($i < j$) και αν είναι επίσης και μεγαλύτερη από την τιμή $K[j]$ ($K[j] < i$). Με άλλα λόγια ελέγχει αν είναι μεγαλύτερος αριθμός που βρίσκεται αριστερότερα, αλλά δεξιότερα από κάθε άλλο μεγαλύτερο (μέχρι την δεδομένη στιγμή).
Αν ισχύει αυτό, ενημερώνουμε το $K[j] \leftarrow i$

Πολυπλοκότητα

Το Βήμα 1 χρειάζεται $O(n)$, το Βήμα 2 χρειάζεται $O(2n) = O(n)$ και το Βήμα 4 χρειάζεται $O(3n \log n) = O(n \log n)$.

Επομένως, ο συνολικός αλγόριθμος χρειάζεται $O(n \log n)$

Απόδειξη ορθότητας

Θα δείξουμε την ορθότητα με επαγωγή στο μήκος του πίνακα.

Επαγωγική Βάση: Για $n = 1$

Ισχύει τετριμμένα, αφού από η αρχικοποίηση ήταν στο 0 και η παραδοχή είναι ότι το πρώτο στοιχείο κυριαρχείται από το 0.

Επαγωγική Υπόθεση: Έστω ότι ισχύει για $n < k$ όπου, $2 \leq k \in \mathbb{N}$.

Επαγωγικό Βήμα: Π.ν.δ.ο. ισχύει για $n = k$.

Θεωρούμε τώρα τον υπο-πίνακα $A' = A[1, \dots, n-1]$.

Παρατηρούμε ότι από επαγωγική υπόθεση ο αλγόριθμος μας επιστρέφει σωστά τα τις κυρίαρχες δέσεις (πίνακα K).

Ξανατρέχουμε τον αλγόριθμο για τον πίνακα $A'' = [A', A[k]]$.

Παρατηρούμε αρχικά ότι τρέχοντας τον αλγόριθμο, οι τιμές του πίνακα K (για τα πρώτα $n-1$ στοιχεία) δεν αλλάζουν - όπως άλλωστε θα δέλαμε γιατί το $A[k]$ βρίσκεται δεξιότερα από όλα τα υπόλοιπα στοιχεία.

Όσο αφορά το στοιχείο $A[k]$ θα ταξινομηθεί στο πίνακα A'' όπου περιέχει τον ταξινομημένο A' . Το στοιχείο $A[k]$ θα πάρει μια νέα θέση έστω j , αυτό σημαίνει ότι συγκρίθηκε με τα τελευταία $n-j$ στοιχεία του A' τα οποία ήταν όλα μεγαλύτερά του. Επομένως για κάθε μεγαλύτερο του στοιχείο, ελέγχθηκε αν η θέση του ήταν μεγαλύτερη από την εκάστοτε $K[k]$.

Οπότε το $K[k]$ πήρε σωστή τιμή (αφού συγκρίθηκε με όλα τα μεγαλύτερα και ελέγχθηκε αν βρισκόταν αριστερότερα και η θέση τους ήταν μεγαλύτερη από την $K[k]$ που αρχικά ήταν 0).

Άσκηση 2: Επιλογή

Εκφώνηση

- (α) Έστω πολυσύνολο (multiset) S με n θετικούς ακέραιους που όλοι είναι μικρότεροι ή ίσοι δεδομένου ακεραίου M . Έχουμε πρόσβαση (μόνο) στην κατανομή F_S των στοιχείων της συλλογής. Συγκεκριμένα, έχουμε στη διάθεσή μας συνάρτηση $F_S(\ell)$ που για κάθε φυσικό ℓ , επιστρέφει το πλήθος των στοιχείων του S που δεν ξεπερνούν το ℓ , δηλ. $F_S(\ell) = \{x \in S : x \leq \ell\}$. Να διατυπώσετε αποδοτικό αλγόριθμο που δέχεται ως είσοδο φυσικό k , $1 \leq k \leq n$, και υπολογίζει (καλώντας την F_S) το k -οστό μικρότερο στοιχείο του S . Να αιτιολογήσετε την ορθότητα του αλγορίθμου σας και να προσδιορίσετε το πλήθος των απαιτούμενων κλήσεων στην F_S (στη χειρότερη περίπτωση). Προσπαθήστε το πλήθος των κλήσεων στην F_S να μην εξαρτάται από το n (μπορεί όμως να εξαρτάται από το M).
- (6) Έστω πίνακας διαφορετικών θετικών ακεραίων $A[1 \dots n]$ και έστω M το μέγιστο στοιχείο του A . Θεωρούμε το πολυσύνολο S που αποτελείται από όλες τις μη αρνητικές διαφορές ζευγών στοιχείων του A . Δηλαδή, έχουμε:

$$S = \{A[i] - A[j] \mid i \neq j \text{ και } A[i] > A[j]\}$$

Να διατυπώσετε αποδοτικό αλγόριθμο που υπολογίζει το k -οστό μικρότερο στοιχείο του S . Να προσδιορίσετε την υπολογιστική πολυπλοκότητα του αλγορίθμου σας (συναρτήσει των n και M) και να αιτιολογήσετε την ορθότητά του.

Υπόδειξη: Υλοποιείτε αποδοτικά την F_S και χρησιμοποιείτε τον αλγόριθμο του (α).

Λύση: Ερώτημα α)

Διαισθητική εξήγηση

Δεδομένου ότι θα πρέπει να υπολογίσουμε την τιμή του k -οστού μικρότερου στοιχείου ενός πεπερασμένου πολυσυνόλου S , χωρίς να έχουμε πρόσβαση στα στοιχεία του θα πρέπει να αξιοποιήσουμε την F_S και το άνω φράγμα M . Αφού η $F_S(l)$ μας επιστρέφει το πλήθος των στοιχείων που είναι μικρότερα από l μας δίνει μια έννοια “διάταξης”. Η διάταξη αυτή οφείλεται στο γεγονός ότι η F_S είναι αύξουσα, αυτό μας οδηγεί σε ένα αλγόριθμο ο οποίος θα μοιάζει ως ιδέα με το *Binary Search*, αφού εκεί ζητούσαμε τα στοιχεία να είναι διατεταγμένα ως προς την σχέση διάταξης “ $\leq$ ”.

Θα λύσουμε το πρόβλημα κάνοντας κάποιες υποθέσεις για την τιμή του k -οστού μικρότερου τις οποίες θα βελτιώνουμε. Αρχικά υποθέτουμε ότι ο αριθμός θα είναι μικρότερος ή ίσος του άνω φράγματος (M) και μεγαλύτερος ή ίσος της μονάδας (1). Θα μαντεύουμε ότι ο αριθμός βρίσκεται στην μέση αυτών των 2 (i) και θα ελέγχουμε πόσο κοντά έχουμε πέσει. Αν το πλήθος $F_S(i)$ είναι μεγαλύτερο ή ίσο του k τότε είναι λογικό να υποθέσουμε ότι ο αριθμός είναι τουλάχιστον 1 και το πολύ i . Αλλιώς (αν είναι γνήσια λιγότερο) συμπεραίνουμε ότι ο αριθμός μας είναι τουλάχιστον $i + 1$ και το πολύ M .

Αλγόριθμος

Για να λύσουμε το πρόβλημά μας, καλούμε την συνάρτηση με ορίσματα $L \leftarrow 1, R \leftarrow M$.

BS_Variation(L, R)

1. Αν $L = R$, επέστρεψε L (ή συμμετρικά R)
2. $i \leftarrow \lfloor \frac{L+R}{2} \rfloor$
3. Αν $F_S(i) \geq k$ τότε *BS_Variation*(L, i)
4. Αν $F_S(i) < k$ τότε *BS_Variation*($i + 1, R$)

Ορθότητα

Η ορθότητα του αλγορίθμου γίνεται με επαγωγή στο μέγεθος του άνω φράγματος.

Επαγωγική Βάση: Για $M = 1$

Τότε το $L = M = 1$, επομένως όλα τα στοιχεία είναι 1, άρα θα επιστρέψει $L = 1$ το οποίο είναι ορθό.

Επαγωγική Υπόθεση: Έστω ότι ισχύει για $M < k$ όπου, $2 \leq k \in \mathbb{N}$.

Επαγωγικό Βήμα: Π.ν.δ.ο. ισχύει για $M = k$.

Προφανώς τώρα, $L \neq M$. Επομένως το $i \leftarrow \lfloor \frac{L+R}{2} \rfloor$.

Αν το πλήθος των αριθμών που είναι μικρότεροι ή ίσοι από i είναι μεγαλύτερο του k , τότε μας αρκεί να ψάξουμε τιμές μικρότερες ή ίσες του i (γιατί σίγουρα το k -οστό μικρότερο θα είναι το πολύ i). Καλώντας ξανά την συνάρτηση από Ε.Υ. θα πάρουμε το σωστό αποτέλεσμα.

Αν το πλήθος των αριθμών που είναι μικρότεροι ή ίσοι από i είναι (γνήσια) μικρότερο του k , τότε ο αριθμός μας θα είναι τουλάχιστον $i + 1$. Από επαγωγική υπόθεση το αποτέλεσμα θα είναι σωστό.

Πολυπλοκότητα

Ο αλγόριθμος εκτελεί $O(\log M)$ κλήσεις της F_S .

Λύση: Ερώτημα 6)

Παρατηρήσεις - Μεθοδολογία

Παρατηρούμε ότι: $S = \{A[i] - A[j] : 0 < A[i] - A[j]\}$.

Επομένως $F_S(l) = |\{A[i] - A[j] : 0 < A[i] - A[j] \leq l\}|$.

Επίσης το M , αφού αποτελεί τιμή μέγιστου στοιχείου, αποτελεί και άνω φράγμα.

Συνεπώς απο ερώτημα (α) αρκεί να υλοποιήσουμε την F_S .

Αλγόριθμος

- Ταξινομούμε τον πίνακα A κατά αύξουσα σειρά
- Υλοποίηση $F_S(k)$
 1. Αρχικοποιούμε ένα μετρητή στο 0 ($s \leftarrow 0$)
 2. Για κάθε στοιχείο $i = 1$ έως n
 - (α') Εκτελούμε *Binary Search* στον (ταξινομημένο πίνακα) A ώστε να βρούμε τον μικρότερο δείκτη j του οποίου το στοιχείο $A[j]$ έχει διαφορά μέχρι και k από το $A[i]$ ($A[i] - A[j] \leq k$)
 - (β') Αυξάνουμε τον μετρητή s κατά $j - i$ ($s \leftarrow s + (j - i)$)
- Εκτελούμε την $BS_Variation(0, M)$, όπου M , το μέγιστο στοιχείο (δίνεται)

Πολυπλοκότητα

Ο συνολικός χρόνος του αλγορίθμου θα είναι $O(n \log n \log M + n \log n) = O(n \log n \log M)$, αφού θα εκτελεστούν $O(\log M)$ κλήσεις της F_S η οποία για ταξινομημένο πίνακα χρειάζεται χρόνο $O(n \cdot \log n)$.

Ορθότητα

Α.ν.δ.ο. ο αλγόριθμος F_S που σχεδιάσαμε είναι ορθός.

Για κάθε στοιχείο i βρίσκουμε τον μικρότερο δείκτη j στον ταξινομημένο (κατά αύξουσα σειρά) πίνακα A (εννοείται $j < i$) ώστε $A[i] - A[j] \leq k$.

Αυτό σημαίνει ότι κάθε δείκτης j' μικρότερος του j δεν θα έχει τιμή τέτοια ώστε $A[i] - A[j'] \leq k$ και κάθε $j'' : j < j'' < i$ θα ικανοποιεί επίσης την συνθήκη $A[i] - A[j''] \leq k$.

Οπότε για κάθε στοιχείο το πλήθος των στοιχείων που έχουν θετική διαφορά και μικρότερη ή ίση με το k είναι ακριβώς τα στοιχεία από j μέχρι $i - 1$ επομένως $i - j$ το πλήθος.

Για κάθε στοιχείο αυξάνουμε τον μετρητή κατά το $i - j(i)$ (το j κάθε φορά εξαρτάται από το i).

Άσκηση 3: Άθροισμα Στοιχείων Υποσυνόλων και Υπακολουδιών

Εκφώνηση

- (α) Θεωρούμε σύνολο $S = \{s_1, s_2, \dots, s_n\}$ με $n \geq 3$ θετικούς ακέραιους. Για δεδομένους ακραίους $B \geq 1$ και $k \geq 1$, θέλουμε να υπολογίσουμε το πλήθος των υποσυνόλων $A \subseteq S$ με $|A| \leq k$ στοιχεία και άθροισμα στοιχείων ίσο με B , δηλαδή $\sum_{s_i \in A} s_i = B$. Να διατυπώσετε αποδοτικό αλγόριθμο για αυτό το πρόβλημα, και να αιτιολογήσετε την ορθότητα και την υπολογιστική του πολυπλοκότητα. Είναι ο αλγόριθμος που προτείνετε πολυωνυμικού χρόνου (εξηγήστε);
- (β) Θεωρούμε ακολουθία $\alpha = (\alpha_1, \dots, \alpha_n)$ αποτελούμενη από $n \geq 3$ θετικούς ακραίους αριθμούς. Για δεδομένο ακέραιο $k, 1 \leq k \leq n-1$, θέλουμε να υπολογίσουμε μια k -σχεδόν γνησίως αύξουσα υπακολουθία της α , τα στοιχεία της οποίας επιτυγχάνουν μέγιστο συνολικό άθροισμα (μεταξύ όλων των k -σχεδόν γνησίως αυξουσών c της α). Μια (υπακ)ακολουθία $(\beta_1, \beta_2, \dots, \beta_n)$ είναι k -σχεδόν γνησίως αύξουσα αν η ιδιότητα ότι η $(\beta_1, \beta_2, \dots, \beta_n)$ είναι γνησίως αύξουσα “διακόπτεται” σε k σημεία το πολύ, δηλ. αν υπάρχουν k δείκτες $\{i_1, \dots, i_k\} \subseteq \{1, \dots, n-1\}$ ώστε να ισχύει ότι για κάθε $i \in \{1, \dots, n-1\} \setminus \{i_1, \dots, i_k\}, \beta_i < \beta_{i+1}$. Να διατυπώσετε αποδοτικό αλγόριθμο για αυτό το πρόβλημα. Να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα του αλγορίθμου σας.

Λύση: Ερώτημα α)

Περιγραφή

Θυμόμαστε το πρόβλημα του αθροίσματος υποσυνόλου, όπου για δοθέν σύνολο, υπολογίζουμε το πλήθος των υποσυνόλων των οποίων τα στοιχεία αθροίζουν σε ένα συγκεκριμένο αριθμό.

Η λύση του προβλήματος αυτού ήταν με χρήση δυναμικού προγραμματισμού ο οποίος είχε σύνολο καταστάσεων πλήθους δύο.

Τώρα επειδή προφανώς έχουμε μια επιπλέον παράμετρο θα πρέπει να αυξήσουμε το σύνολο καταστάσεων (state space) κατά ένα βάζοντας και τον νέο περιορισμό, το k .

Αλγόριθμος

- Θεωρούμε ένα πίνακα $DP[i, j, l] = [0, \dots, 0]$, ο οποίος κρατάει το μέγιστο άθροισμα από τα πρώτα $i, i = 0, \dots, n$ στοιχεία, χρησιμοποιώντας ακριβώς $l, l = 0, \dots, k$ από αυτά και τα στοιχεία τους αθροίζουν στο $j, j = 0, \dots, B$.
- Αρχικοποιούμε τις εξής τιμές:
 - $DP[0, 0, 0] = 1$ (υπάρχει ένας τρόπος να πάρουμε άθροισμα 0 από σύνολο με 0 στοιχεία)
 - $DP[0, j, l] = 0 \quad \forall j, l \geq 1$
 - $DP[i, j, l] = 0 \quad \forall l > i$
- $$DP[i, j, l] = DP[i-1, j, l] + \begin{cases} DP[i-1, j-s_i, l-1], & \text{αν } j \geq s_i \text{ και } l \geq 1, \\ 0, & \text{αλλιώς.} \end{cases}$$
- Αφού συμπληρώσουμε τον πίνακα η τελική απάντηση δίνεται από: $\sum_{c=1}^k DP[n, B, c]$, δηλαδή από n διαδέσιμα στοιχεία που αθροίζουν στο B , επιλέγοντας ακριβώς c στοιχεία.

Πολυπλοκότητα

Συμπληρώνουμε κάθε στοιχείο του πίνακα σε σταθερό χρόνο και υπολογίζουμε συνολικά nBk στοιχεία. Τα υπόλοιπα βήματα δεν προσφέρουν στην τελική πολυπλοκότητα του αλγορίθμου. Οπότε η υπολογιστική (χρονική) πολυπλοκότητα είναι $O(nBk)$.

Ο αλγόριθμος δεν είναι πολυωνυμικού χρόνου αλλά ψεύδο-πολυωνυμικού χρόνου.

Αυτό ισχύει διότι το μήκος της κωδικοποίησης του B είναι λογαριθμικό ως προς την τιμή του (αντίστοιχα η τιμή του είναι εκθετική ως προς το μήκος του).

Οπότε η χρονική πολυπλοκότητα του αλγορίθμου θα είναι εκθετικού μεγέθους ως προς το μήκος της εισόδου του.

Ορθότητα

Σε κάθε βήμα έχουμε ένα επιπλέον στοιχείο στην διάδεσή μας, το i -οστό (s_i).

Αν μπορούμε να το πάρουμε τότε, αυξάνουμε το πλήθος κατά το πλήθος των υποσυνόλων με $l - 1$ διαθέσιμα που αθροίζουν στο $j - s_i$ ώστε να υπολείπεται το s_i για να βγεί το άθροισμα K . Αλλιώς, αν δεν μπορούμε να το χρησιμοποιήσουμε επειδή δεν έχουμε άλλα διαθέσιμα αντικείμενα (ξεπεράσαμε το k) ή δεν μπορούμε να το πάρουμε $j < s_i$, προσθέτουμε επειδή 0 στο πλήθος των υποσυνόλων με διαθέσιμα τα πρώτα $i - 1$ στοιχεία.

Οπότε έτσι, καλύπτουμε όλα τα δυνατά ενδεχόμενα.

Λύση: Ερώτημα 6)

Περιγραφή

Γνωρίζουμε πως να υπολογίζουμε αύξουσα υπακολουθία με μέγιστο άθροισμα.

Τώρα θέλουμε να υπολογίσουμε k -σχεδόν αύξουσα υπακολουθία.

Θα αυξήσουμε τον χώρο των καταστάσεων κατά ένα, βάζοντας το k .

Προφανώς για $k = 0$ προκύπτει το πρόβλημα αύξουσας υπακολουθίας με μέγιστο άθροισμα.

Αλγόριθμος

Έστω $\alpha = (\alpha_1, \dots, \alpha_n)$ και $k \in \mathbb{N}$

1. Αρχικοποιούμε ένα πίνακα $DP[i, j] \in \mathbb{N}^{n+1 \times k+1}$, $i = 0, \dots, n$, $j = 0, \dots, k$ όπου αντιστοιχεί στο μέγιστο άθροισμα που τελειώνει στο της α_i μιας j -σχεδόν αύξουσας υπακολουθίας.

(α') Κάθε $DP = [0, j] = 0$ και για να γεμίσουμε τον $DP = [i, 0]$ χρησιμοποιούμε τον γνωστό αλγόριθμο (με δυναμικό προγραμματισμό).

2. Η σχέση για τα υπόλοιπα i, j είναι η εξής:

$$DP[i, j] = \max\{\alpha_i + \max_{j < i, A[j] < A[i]} DP[j, k], \alpha_i + \max_{j < i, A[j] \geq A[i]} D[j, k - 1]\}$$

3. Αν θέλουμε να υπολογίσουμε την υπακολουθία αυτή, κρατάμε για κάθε στοιχείο του πίνακα το j που μας έδωσε αυτό το $\max$, δηλαδή το στοιχείο του οποίου διαλέξαμε ως προηγούμενο. Επομένως αρχικοποιούμε ακόμη ένα $n \times k$ πίνακα έστω $Prev$ τέτοιος ώστε:

$$Prev[i, j] = \arg \max_j \{\alpha_i + \max_{j < i, A[j] < A[i]} DP[j, k], \alpha_i + \max_{j < i, A[j] \geq A[i]} D[j, k - 1]\}$$

4. Στο τέλος το μέγιστο άθροισμα το δίνει $\max_i \{DP[i, k]\}$ και η ακολουθία είναι η:
($f := \arg \max_i \{DP[i, k]\}$, $Prev[f]$, $Prev[Prev[f]]$, ...)

Πολυπλοκότητα

Η πολυπλοκότητα του αλγορίθμου είναι $O(n^2k)$ γιατί, για κάθε στοιχείο από τα nk συνολικά χρειάζεται n χρόνο για να υπολογιστεί (λόγω διαπέρασης της προηγούμενης και προ-προηγούμενης γραμμής μία φορά).

Ορθότητα

Η παρακάτω απόδειξη της ορθότητας κρύβει μια επαγωγή (ουσιαστικά αποτελεί την εξήγηση στο επαγωγικό βήμα).

Ο αλγόριθμος εξετάζει κάθε φορά το μέγιστο άθροισμα μιας l -σχεδόν αύξουσας υπακολουθίας που τελειώνει στον όρο j . Οι επιλογές σε κάθε περίπτωση είναι: είτε να παίρνουμε το στοιχείο a_i και βρίσκουμε το μέγιστο άθροισμα που του “ταιριάζει” ώστε να μην χρεωθούμε “παράβαση”, ή το παίρνουμε και να χρεωθείς μια “παράβαση” της αύξουσας ακολουθίας, όπου στην περίπτωση αυτή αρκεί να βρούμε το μέγιστο άθροισμα μέχρι τώρα (που προφανώς δεν θα “ταιριάζει με το a_i γιατί αλλιώς θα ήμασταν στην πρώτη περίπτωση) και να το αυξήσουμε κατά a_i .

Εδώ η μόνη περίπτωση που χρειάζεται λίγο προσοχή είναι οι αρχικοποιήσεις: για $l = 0$ έχουμε το πρόβλημα του μέγιστου αθροίσματος αύξουσας υπακολουθίας και για $i = 0$ η απάντηση είναι πάντα 0 γιατί δεν ελέγχουμε κανένα στοιχείο (η ακολουθία ξεκινάει με δείκτη ένα) οπότε το άθροισμα θα είναι πάντα μηδενικό.

Σημείωση: Όπου αναφέραμε “ταιριάζει” εννοούμε ότι δεν χρεωνόμαστε “παράβαση”, όπου παράβαση εννοούμε όταν επιλέγουμε ένα όρο που δεν είναι μεγαλύτερος από τους προηγούμενους (σπάει την συνθήκη αύξουσας επιλογής των όρων).

Άσκηση 4: Μη Επικαλυπτόμενα Διαστήματα Μέγιστου Συνολικού Μήκους

Εκφώνηση

Θεωρούμε n διαστήματα $[s_1, f_1), \dots, [s_n, f_n)$ στην ευθεία των φυσικών αριθμών (έχουμε λοιπόν ότι $s_i, f_i \in \mathbb{N}$ και $s_i < f_i$, για κάθε $1 \leq i \leq n$). Θέλουμε να επιλέξουμε κάποια από αυτά τα n διαστήματα, ώστε τα επιλεγμένα διαστήματα να μην επικαλύπτονται μεταξύ τους και να έχουν μέγιστο συνολικό μήκος. Το μήκος ενός διαστήματος $[s_i, f_i)$ είναι ίσο με $f_i - s_i$.

1. Να δείξετε, μέσω αντιπαραδείγματος, ότι ο άπληστος αλγόριθμος που επιλέγει, σε κάθε επανάληψη, το διαθέσιμο διάστημα $[s_i, f_i)$ με μέγιστο μήκος $f_i - s_i$ δεν οδηγεί απαραίτητα στη βέλτιστη λύση. Να επαναλάβετε για τον άπληστο αλγόριθμο που επιλέγει το διαθέσιμο διάστημα $[s_i, f_i)$ με ελάχιστο χρόνο ολοκλήρωσης f_i .
2. Να διατυπώσετε αποδοτικό αλγόριθμο για το πρόβλημα της επιλογής μη επικαλυπτόμενων διαστημάτων με μέγιστο συνολικό μήκος. Να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα του αλγορίθμου.

Λύση: Ερώτημα 1.

Κριτήριο: μέγιστο μήκος

Θεωρούμε τα διαστήματα $[1, 4)$, $[1, 3)$, $[3, 5)$.

Ο άπληστος αλγόριθμος που επιλέγει, σε κάθε επανάληψη, το διαθέσιμο διάστημα $[s_i, f_i)$ με μέγιστο μήκος $f_i - s_i$ σε αυτή τη περίπτωση θα επιλέξει αρχικά το $[1, 4)$ μήκους 3 και έπειτα θα τερματίσει χωρίς να επιλέξει άλλο διάστημα (αφού το διάστημα $[1, 3)$ ξεκινάει ταυτόχρονα αλλά έχει μικρότερο μήκος και το $[3, 5)$ δεν ξεκινάει πριν τελειώσει το $[1, 4)$).

Το συνολικό μήκος των διαστημάτων που επιλέγει αυτός ο αλγόριθμος είναι $4 - 1 = 3$.

Παρατηρούμε ότι αυτό πράγματι δεν αποτελεί βέλτιστη λύση, αφού η επιλογή των $[1, 3)$, $[3, 5)$ δίνει συνολικό μήκος $(3 - 1) + (5 - 3) = 4 > 3$.

Κριτήριο: νωρίτερη περάτωση

Θεωρούμε τα διαστήματα $[1, 3)$ και $[2, 5)$.

Ο άπληστος αλγόριθμος που επιλέγει, σε κάθε επανάληψη, το διαθέσιμο διάστημα $[s_i, f_i)$ με ελάχιστο f_i , σε αυτή τη περίπτωση, θα επιλέξει αρχικά το $[1, 3)$ μήκους 2 και έπειτα θα τερματίσει χωρίς να επιλέξει άλλο διάστημα (αφού το διάστημα $[2, 5)$ δεν ξεκινάει πριν τελειώσει το $[1, 3)$).

Το συνολικό μήκος των διαστημάτων που επιλέγει αυτός ο αλγόριθμος είναι $3 - 1 = 2$.

Παρατηρούμε ότι αυτό πράγματι δεν αποτελεί βέλτιστη λύση, αφού η επιλογή του $[2, 5)$ δίνει συνολικό μήκος $(5 - 2) = 3 > 2$.

Λύση: Ερώτημα 2.

Περιγραφή - Διαισθητική προσέγγιση

Παρατηρούμε ότι είναι δύσκολο να βρούμε ένα άπληστο κριτήριο, αφού πολλές φορές η βέλτιστη επιλογή δεν μπορεί να αποφασιστεί αν δεν γνωρίζουμε και τα επόμενα διαστήματα.

Αλγόριθμος 1

1. Ταξινόμησε κατά αύξουσα σειρά τα f_j
2. Αρχικοποίησε ένα πίνακα $D = [(0, f_1), \dots, (0, f_r)]$ μήκους όσο το πλήθος των διαφορετικών f_j (το πολύ n), (δηλ. $r = |\{f_j : j = 1, \dots, n\}|$)
όπου κάθε ζεύγος $D[i] = (d_i, t_i)$ αναπαριστά την βέλτιστη τιμή d_i μέχρι την χρονική στιγμή t_i .
*Ο πίνακας είναι ταξινομημένος ως προς την δεύτερη συντεταγμένη, οπότε για αναζήτηση μπορούμε να χρησιμοποιούμε **Binary Search***
3. Για κάθε διάστημα $j \leftarrow 1$ έως n , ενημερώνουμε την τιμή του πίνακα D ως εξής:
 - (α') Εκτελώντας δυαδική αναζήτηση βρίσκουμε τον δείκτη k ώστε $t_k = f_j$ (τον δείκτη της θέσης του πίνακα που αναφέρεται στην βέλτιστη λύση μέχρι το f_j) και τον δείκτη k' ώστε $t_{k'} \leq s_j$ και $t'_k \geq m$, $\forall m \leq s_j$ (ο μεγαλύτερος δείκτης ώστε η τιμή της δεύτερης συντεταγμένης να είναι μικρότερη από το f_j)
 - (β') $d_k \leftarrow \max\{d_k, d_{k-1}, d_{k'} + (f_j - s_i)\}$
Έχουμε δηλαδή τρία ενδεχόμενα, με την σειρά που τα βάλαμε στο $\max$ είναι τα εξής: να μην αξίζει να αλλάξουμε την τιμή (γιατί μπορεί να υπάρχει άλλο διάστημα που τελειώνει ίδια ώρα δίνει μεγαλύτερο άθροισμα), αξίζει να μην το βάλουμε και αξίζει να το βάλουμε

Αλγόριθμος 2

1. Ταξινόμησε κατά αύξουσα σειρά τα διαστήματα βάσει του f_i
2. Αρχικοποίησε ένα πίνακα $D = [0, \dots, 0]$ μήκους $f_n + 1$,
ο οποίος θα αναπαριστά την βέλτιστη λύση $D[j]$ μέχρι την χρονική στιγμή j .
3. Ξεκινάμε με το πρώτο διάστημα ($i \leftarrow 1$)
4. Ενημερώνουμε για κάθε $j \leftarrow 1$ έως f_n την τιμή του $D[j]$ ως εξής:
 - (α') Αν $j \neq f_i$, τότε $D[j] \leftarrow D[j - 1]$
 - (β') Αν $j = f_i$, τότε $D[j] \leftarrow \max\{D[j], D[j - 1], D[s_i] + (f_i - s_i)\}$ (έλεγε αν αξίζει να προστεθεί) και $i \leftarrow i + 1$ (προχώρα στο επόμενο διάστημα)
 - i) Αν το επόμενο διάστημα τελειώνει την ίδια στιγμή, ελέγχεται πριν το j αυξηθεί κατά 1
 - ii) Εννοείτε αν το i ξεπεράσει n δεν θα ελέγξει κάποιο άλλο διάστημα

Ο αλγόριθμος αυτός εκτελείται σε $O(f_n)$ αλλά είναι ψεύδο-πολυωνυμικός.

Όμως, σε περιπτώσεις όπου έχουμε μικρή μέγιστη περάτωση και πάρα πολλά διαστήματα σε στο μικρό αυτό διάστημα $[1, f_n]$ ενδέχεται να λειτουργήσει καλύτερα.

Πολυπλοκότητα

Όσο αφορά τον Αλγόριθμο 1, εκτελούμε ταξινομήση σε $O(n \log n)$, αρχικοποιούμε τον πίνακα σε $O(2n) = O(n)$, και για κάθε διάστημα εκτελούμε δυαδική αναζήτηση και κάνουμε συγκρίσεις $O(2 \cdot n \cdot \log n)$.

Επομένως ο συνολικός χρόνος είναι $O(n \log n)$.

Ορθότητα

Με επαγωγή στο πλήθος των διαστημάτων n .

Στο επαγωγικό βήμα θα επιχειρηματολογήσουμε ως εξής:

Ο αλγόριθμος είναι ορθός διότι, καλύπτουμε όλο το δυνατό εύρος επιλογών αφού ο πίνακας του δυναμικού προγραμματισμού αποθηκεύει το μέγιστο μήκος για κάθε χρόνο περάτωσης ο οποίος προκύπτει

από τα διαστήματα.

Οι δυνατές επιλογές είναι: Να έχουμε βρει (ήδη) διάστημα που τελειώνει στην ίδια ώρα και να μην αξίζει να πάρουμε αυτό που εξετάζουμε (οπότε κρατάμε αυτό), (να μην έχουμε βρει ήδη διάστημα που τελειώνει ίδια ώρα και) να μην αξίζει να το επιλέξουμε και τέλος να αξίζει (είτε έχουμε εξετάσει αλλά που τελειώνουν ίδια ώρα είτε όχι) να το επιλέξουμε. Οι οποίες ελέγχονται όλες.

Άσκηση 5: Θέσεις Στάδμευσης

Εκφώνηση

Έχουμε N αυτοκίνητα που θέλουν να παρκάρουν κατά μήκος του διαστήματος $(0, L]$ (σκεφτόμαστε το διάστημα $(0, L]$ ως μια ευθεία οδό μήκους L όπου τα αυτοκίνητα παρκάρουν μόνο στη μία πλευρά). Κάθε αυτοκίνητο i χαρακτηρίζεται από μια τριάδα $C_i = (v_i, d_i, s_i)$ θετικών ακεραίων, όπου το v_i δηλώνει αξία, το d_i μέγιστη απόσταση από το 0 και το s_i το μήκος του αυτοκινήτου.

Το αυτοκίνητο i προσφέρει v_i ευρώ για να παρκάρει σε διάστημα $(t, t + s_i]$ μήκους s_i με $t + s_i \leq d_i$ (χβτγ. θεωρούμε πως η αρχή κάθε διαστήματος είναι ακέραιος). Δηλαδή, το αυτοκίνητο i δέχεται να παρκάρει (και προσφέρει v_i ευρώ) μόνο σε διάστημα μήκους s_i που το τέλος του δεν ξεπερνά το σημείο d_i . Το αυτοκίνητο i δεν δέχεται να παρκάρει σε διάστημα που έχει μήκος μικρότερο από s_i ή τελειώνει μετά το d_i . Το ζητούμενο είναι να επιλέξουμε ποια αυτοκίνητα θα παρκάρουν και σε ποια διαστήματα, έτσι ώστε: (i) σε κάθε σημείο του $(0, L]$ να παρκάρει το πολύ ένα αυτοκίνητο, και (ii) να μεγιστοποιηθεί η συνολική αξία των αυτοκινήτων που παρκάρουν σύμφωνα με τις επιθυμίες τους. Να διατυπώσετε αποδοτικούς αλγόριθμους για αυτό το πρόβλημα, και να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα των αλγορίθμων σας, σε καθεμιά από τις παρακάτω περιπτώσεις:

1. Έχουμε $s_i = 1$ για κάθε αυτοκίνητο i (τα v_i και d_i είναι θετικοί ακέραιοι χωρίς περιορισμούς). Θεωρούμε δηλαδή την ειδική περίπτωση όπου τα αυτοκίνητα έχουν μοναδιαίο μήκος και αντιμετωπίζουμε το διάστημα $(0, L]$ ως σύνολο L διαδέσιμων θέσεων, με την ℓ -οστή θέση, $\ell = 1, \dots, L$, να αντιστοιχεί στο διάστημα $(\ell - 1, \ell]$. Το αυτοκίνητο i συνεισφέρει αξία v_i αν παρκάρει στη θέση $\ell \leq d_i$, και αξία 0 διαφορετικά.
2. Θεωρούμε τη γενική περίπτωση όπου τα v_i, d_i και s_i είναι θετικοί ακέραιοι χωρίς περιορισμούς.

Λύση: Ερώτημα 1.

Σκιαγράφιση Απόδειξης

Θέλουμε να μεγιστοποιήσουμε το κέρδος. Θέλουμε δηλαδή να επιλέξουμε διαστήματα (υπό σωστές προϋποθέσεις) τέτοια ώστε το άθροισμα των αξιών τους να είναι μέγιστο.

Αυτό που “χάνουμε” μετά από κάθε παρκάρισμα είναι χώρος μήκους 1 για κάθε αμάξι (μία θέση από τις L συνολικές). Οπότε για κάθε διάστημα το “αντάλλαγμα” που δίνουμε είναι σταθερό.

Αυτό μας προϊδεάζει ότι ο αλγόριθμος θα επιλέγει αμάξια βάσει άπληστου κριτηρίου.

Αλγόριθμος

Θεωρούμε το $(0, L]$ ως σύνολο διακριτών θέσεων, έστω $S = [1, \dots, L]$ και

Θεωρούμε ένα πίνακα $C = [(v_1, d_1), \dots, (v_N, d_N)]$, όπου κάθε θέση αντιστοιχεί στα στοιχεία του αυτοκινήτου.

1. Ταξινόμηση του C ως προς τη δεύτερη συντεταγμένη (αξία) κατά φθίνουσα σειρά και τις ισότητες κατά αύξουσα της πρώτης συντεταγμένης (deadline).
2. Δημιούργησε πίνακα $M[1, \dots, L] = [0, \dots, 0]$
3. Για κάθε αμάξι $i \leftarrow 1$ έως N :
Θα γεμίσουμε τον πίνακα S από δείκτες αυτοκινήτων που επιλέξαμε.
 - (α') Αν γίνεται τοποθέτηση το αμάξι δείκτης i στην μεγαλύτερη δυνατή θέση του πίνακα που είναι μικρότερη από το d_i .
 - (β') Αν δεν υπάρχει επέλεξε το επόμενο αμάξι.
 - (γ') Αν δεν υπάρχει επόμενο αμάξι, ο αλγόριθμος ολοκληρώνεται.

Πολυπλοκότητα

Η πολυπλοκότητα του αλγορίθμου είναι $O(nL)$, όπου στην (χειρότερη) περίπτωση αντιστοιχεί σε $O(n \cdot n) = O(n^2)$.

Δεν μπορούμε να εκτελέσουμε δυαδική αναζήτηση γιατί δεν έχουμε κάποια ένδειξη σχετικά με το πού μπορεί να βρίσκεται η μεγαλύτερη δέση πριν από το deadline.

Ορθότητα

Συνοπτικά θα μπορούσαμε να χρησιμοποιήσουμε το Επιχείρημα της Ανταλλαγής ως εξής:

Θεωρούμε μια βέλτιστη λύση $c_1, \dots, c_n$.

Έστω $c_k = (v_k, d_k)$ το αυτοκίνητο με την με την μέγιστη τιμή. Διακρίνουμε 2 περιπτώσεις:

Αν έχει παρκαριστεί, τότε προφανώς θα το έχουμε παρκάρει πριν το d_k , θα μπορούμε να το αλλάξουμε με το αμάξι που βρίσκεται στο d_k χωρίς να χάσουμε αξία (αφού το αμάξι που αλλάξουμε θα πάρει ακόμη μικρότερη νέα δέση από την προηγούμενη).

Αν δεν έχει παρκαριστεί, το αντικαθιστούμε με το στοιχείο στην δέση d_k παίρνοντας στην χειρότερη ίδια αξία με πριν.

Επαγωγική Βάση: Για $n = 1$:

Ισχύει τετριμμένα αφού θα επιλέξουμε το μοναδικό αμάξι και θα το βάλουμε στην μοναδική δέση.

Επαγωγική Υπόθεση: Έστω ότι ισχύει για $n < k \in \mathbb{N}$, ($n \geq 2$).

Επαγωγικό Βήμα: Π.ν.δ.ο. ισχύει για $n = k$

Έστω ότι η λύση του αλγορίθμου μας για n διαδέσιμες δέσεις είναι $V(n)$ και η βέλτιστη $V^*(n)$. Έχουμε ότι:

$$\begin{aligned} V(S) &= v_1 + V(S_1), \text{ όπου } S_1 = S[1, \dots, k-1, k+1, \dots, n], k = \min\{d_1, L\} \\ &\geq v_1 + V^*(S_1) \text{ από επαγωγική υπόθεση} \\ &\geq V^*(S) \text{ από επιχείρημα ανταλλαγής} \end{aligned}$$

Στο τελευταίο βήμα ισχυριζόμαστε ότι μπορούμε να αλλάξουμε το στοιχείο στην δέση k της βέλτιστης λύσης χωρίς το συνολικό κέρδος να μειωθεί. Αυτό ισχύει παρατηρώντας τα εξής:

- Δεν επηρεάζουμε το βέλτιστο για τις δέσεις $k+1, \dots, n$
- Το βέλτιστο για τις δέσεις $1, \dots, k$ θα είναι τουλάχιστον όσο καλό ήταν και πριν γιατί:
 - Αν το v_1 είχε επιλεγεί σε μια βέλτιστη λύση πιο νωρίς, μετακινώντας το αργότερα, δεν μειώνεται η βέλτιστη λύση γιατί αυτό θα πάει όσο δεξιότερα γίνεται και το αντικείμενο που ήταν στην δέση του θα πάει πιο αριστερά (οπότε θα συνεχίζει να μας δίνει κέρδος)
 - Αν το v_1 δεν είχε επιλεγεί πιο νωρίς, μπορούμε να το αντικαταστήσουμε με το στοιχείο στο k , αφού θα έχει σίγουρα τιμή μεγαλύτερη ή ίση από αυτό.

Λύση: Ερώτημα 2.

Περιγραφή

Θα χρησιμοποιήσουμε Δυναμικό Προγραμματισμό διότι μπορούμε εύκολα να παρατηρήσουμε ότι για να πάρουμε την βέλτιστη απόφαση μια δεδομένη στιγμή θα πρέπει να ξέρουμε τι συμβαίνει πιο μετά και να μην λειτουργήσουμε άπληστα επιλέγοντας το μέγιστο κέρδος, αφού μπορεί το αμάξι να έχει μεγάλο deadline και να μην “επείγει” η επιλογή του.

Αλγόριθμος

1. Θεωρούμε τον πίνακα $C = [(v_1, d_1, s_1), \dots, (v_N, d_N, s_n)]$
2. Ταξινομούμε ως προς την δεύτερη συντεταγμένη κατά αύξουσα σειρά
3. Θεωρούμε ένα πίνακα $DP = [(0, 0), (0, f_1), \dots, (0, f_k)]$ όπου f_i είναι τα διαφορετικά d_i κατά αύξουσα σειρά, συν 1 (η αρχικοποίησή μας $(0,0)$), $i = 1, \dots, k := |\{d_i : c_i = (v_i, d_i, s_i) \in C\}|$
4. Για κάθε αυτοκίνητο (στοιχείο του C) $i \leftarrow 1$ έως N :
 - (α') Εκτελούμε δυαδική αναζήτηση και βρίσκουμε j τέτοιο $d_i = f_j$
(βρίσκουμε δηλαδή τον δείκτη δέσης του DP που πρέπει να ενημερώσουμε) και j' τ.ω. $f_{j'} - s \leq d_i$ (το μεγαλύτερο δείκτη ώστε να χωράει το αυτοκίνητο να παρκάρει).
 - (β') Υπολογίζουμε το $\max\{(DP[j])_1, (DP[j-1])_1, (DP[j'])_1 + d_i\}$,
όπου με $()_1$ συμβολίζουμε την πρώτη συντεταγμένη. (βλ. Ορθότητα για επεξήγηση)

Πολυπλοκότητα

Ο αλγόριθμος αυτός χρειάζεται $O(n \log n)$ για την ταξινόμηση και για κάθε στοιχείο εκτελεί μια εύρεση μεγίστου $O(1)$ μεταξύ δύο τιμών, οι οποίες δέλουν $O(1)$ και $O(2 \log n) = O(\log n)$ να ανακτηθούν. Επομένως συνολικά ο αλγόριθμος χρειάζεται $O(n \log n)$.

Αν για την εύρεση της των δεικτών στο Βήμα 4(α') χρησιμοποιούσαμε σειριακή αναζήτηση, τότε ο χρόνος θα ήταν $O(n^2)$. Εκμεταλλευόμενοι το γεγονός ότι ο πίνακας είναι ταξινομημένος διότι τα στοιχεία είναι πάντα κατά αύξουσα σειρά, (αφού όσο παραπάνω χώρο έχουμε, τόσο παραπάνω κέρδος μπορούμε να αποκομίσουμε - σίγουρα όχι λιγότερο) εκτελούμε δυαδική αναζήτηση.

Ορθότητα

Ο αλγόριθμος κατ' αρχήν εξετάζει όλα τα αμάξια αν αξίζει να τα παρκάρουμε ή όχι. Δεν υπάρχει κάποιος περιορισμός οπότε ενδέχεται δύο αμάξια να έχουν ίδιο deadline.

Για κάθε αμάξι λοιπόν, ενημερώνουμε την τιμή του πίνακα που αφορά τα αμάξια με deadline όσο αυτού που εξετάζουμε, βάσει 3 ενδεχομένων: να έχουμε εξετάσει αμάξι με ίδιο deadline (νεώτερα) και να μην αξίζει να παρκάρουμε το νέο, (να μην έχουμε εξετάσει αμάξι που τελειώνει ίδια ώρα) και να μην αξίζει να το παρκάρουμε (το νέο) και να αξίζει να το παρκάρουμε (ανεξαιρέτως). Αν αξίζει να το παρκάρουμε τότε προσθέτουμε κατάλληλα την αξία του βρίσκοντας την βέλτιστη τιμή από αμάξια ώστε να "χωράει".

Οπότε ο αλγόριθμος καλύπτει όλα τα ενδεχόμενα υπό όλες τις προϋποθέσεις του προβλήματος.